Data Science / Analyse et Traitement de l'Information

TP-C: LDA, Clustering approaches.

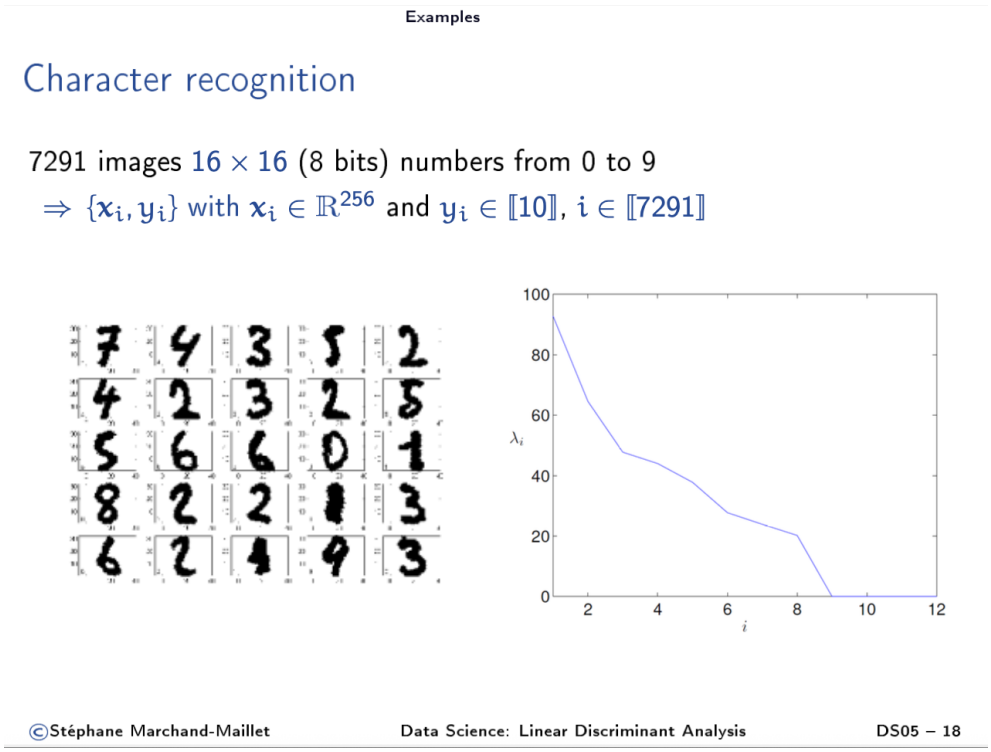
Important: To facilitate this homework, we prepared a Google Colab notebook available at: Colab Notebook Link. Please save a copy to your own Google Drive and edit it directly. You may also write your report and answers directly within the notebook.

1 Task 1: Linear Discriminant Analysis

For LDA part we will work with the `digits` dataset from `sklearn.datasets`:

```
from sklearn.datasets import load_digits
digits = load_digits()
```

The goal of this section is to reproduce results shown in this lecture slide:



1.1 Visualize a few digit images from the dataset.

1.2 Split dataset into train and test datasets. Apply PCA.

1.3 Compute the mean vector of each class.

1.4 Formulate and compute the intra-class scatter matrix S_w.

1.5 Formulate and compute the inter-class scatter matrix S_b.

1.6 Set up the Fisher criterion as a generalized eigenvalue problem:

$$S_b a = \lambda S_w a$$

and solve for eigenvalues and eigenvectors.

1.7 Sort the eigenvalues/eigenvectors and compare with the results shown in the lecture.

1.8 Apply the LDA directions to the test data and visualize in 2D.

1.9 Why do we maximize the ratio of between-class to within-class scatter in LDA?

1.10 For M classes, how many discriminant directions can LDA provide at most?

1.11 What is the key difference between PCA and LDA in terms of the criterion they optimize?

2 Task : k-means

k-means clustering is an unsupervised learning method used to partition data into K clusters.

The algorithm proceeds as follows:

Initialization. Choose the number of clusters K and randomly initialize the centroids $\mu_1, \mu_2, \ldots, \mu_K$.

Assignment step. For each data point x_n, find the nearest centroid μ_i based on the Euclidean distance, and assign the point to that cluster:

$$S_i^{(t)} = \{\, x_n : \|x_n - \mu_i^{(t)}\|^2 \leq \|x_n - \mu_j^{(t)}\|^2,\ \forall j \,\}.$$

Define an indicator variable z_{nk}:

$$z_{nk} = \begin{cases} 1 & \text{if } x_n \text{ belongs to cluster } k, \\ 0 & \text{otherwise.} \end{cases}$$

Update step. Recalculate each centroid as the mean of the data points assigned to it:

$$\mu_i^{(t+1)} = \frac{1}{|S_i^{(t)}|} \sum_{x_j \in S_i^{(t)}} x_j.$$

Convergence. Repeat the assignment and update steps until the centroids stop changing or the change is minimal.

Objective function. K-means minimizes the total within-cluster variance:

$$L = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \|x_n - \mu_k\|^2.$$

Local minima. Because the algorithm depends on the initial centroids, it can converge to a local minimum. Running the algorithm several times with different random initializations and choosing the result with the smallest L helps reduce this problem.

```python
class KMeans:
    def __init__(self, n_clusters, init="random", max_iter=300):
        raise NotImplementedError()

    def fit(self, X, y=None):
        # initialize centroids randomly
        # this function should update self.centroids coordinates
        # for given X
        raise NotImplementedError()

    def predict(self, X):
        # this function for each X returns the cluster
        # it belongs to
        raise NotImplementedError()

```

2.1 Create a subset of MNIST which consists of 2000 randomly sampled instances of "3"s and "7"s.

2.2 Implement k-means from scratch (do not use *sklearn*) with random initialization of the clusters. Perform the algorithm with $K = 2, 3, \ldots, 10$. Compute objective function L defined earlier for each K. Plot L varying with K. Describe the effect of K over L.

2.3 Implement k++ assignment explained in the lecture. Compare the objective function values of several runs with the random and k++ initialization for $K = 2$.

Task 3. Gaussian Mixture Models

Definition

A mixture model assumes that data is generated from a mixture of c components (basis functions) belonging to a family F:

$$f(x) = \sum_{j=1}^{c} \alpha_j \phi(x; \theta_j),$$

where $\alpha_j \geq 0$ and $\sum_j \alpha_j = 1$, and each $\phi(x; \theta_j)$ is controlled by parameters θ_j.

Assumptions

1. The number of components c is known.
2. The family F is known.
3. Labels are unknown.

Density Representation

The mixture can be seen as a process where x is drawn from state ω_j with prior $\Pi(\omega_j)$:

$$f(x) = \sum_{j=1}^{c} \alpha_j \phi(x; \theta_j) = \sum_{j=1}^{c} \Pi(\omega_j)\Pi(x|\omega_j; \theta_j),$$

where $\alpha_j = \Pi(\omega_j)$ and $\phi(x; \theta_j) = \Pi(x|\omega_j; \theta_j)$. Given unlabeled samples $X = \{x_1, \ldots, x_N\}$, estimate parameters $\theta = [\alpha_j, \theta_j]_{j \in [c]}$.

Likelihood and Log-likelihood

$$\Pi(X|\theta) = \prod_{i=1}^{N} \Pi(x_i|\theta), \quad \mathbb{L}(\theta; X) = \sum_{i=1}^{N} \log\left(\sum_{j=1}^{c} \alpha_j \phi(x_i; \theta_j)\right)$$

Gaussian Mixture Model

In a GMM, components are Gaussian:

$$f(x) = \sum_{j=1}^{c} \alpha_j \mathcal{N}(x|\mu_j, \Sigma_j)$$

E-step

Given current parameters $\theta^0 = [\alpha_j^0, \mu_j^0, \Sigma_j^0]_{j\in[c]}$, compute responsibilities:

$$\gamma_{ij}(\theta^0) = \frac{\alpha_j^0 \mathcal{N}(x_i|\mu_j^0, \Sigma_j^0)}{\sum_{k=1}^c \alpha_k^0 \mathcal{N}(x_i|\mu_k^0, \Sigma_k^0)}$$

M-step

Update parameters by weighted maximum likelihood:

$$\hat{\alpha}_j = \frac{1}{N}\sum_i \gamma_{ij}, \quad \hat{\mu}_j = \frac{\sum_i \gamma_{ij} x_i}{\sum_i \gamma_{ij}}, \quad \hat{\Sigma}_j = \frac{\sum_i \gamma_{ij}(x_i - \hat{\mu}_j)(x_i - \hat{\mu}_j)^T}{\sum_i \gamma_{ij}}$$

EM Algorithm for c-component Mixtures

1. Initialize $\theta^0 = [\alpha_j^0, \mu_j^0, \Sigma_j^0]_{j\in[c]}$.
2. E-step: compute γ_{ij}.
3. M-step: update $\alpha_j, \mu_j, \Sigma_j$.
4. Repeat until convergence.

Over-parameterization

In high-dimensional spaces, estimating full covariance matrices ($O(D^2)$ parameters) is difficult. Dimensionality reduction (e.g., PCA) is often applied before fitting GMMs.

```python
class GMM:
    def __init__(self, n_components, max_iter=100, tol=1e-3, reg_covar=1e-6, seed=None):
        self.n_components = int(n_components)
        self.max_iter = int(max_iter)
        self.tol = float(tol)            # stop when llh improvement < tol
        self.reg_covar = float(reg_covar)
        self.seed = seed

        # learned params
        self.pi_ = None                  # mixing weights (K,)
        self.means_ = None               # means (K, D)
        self.covariances_ = None         # covariances (K, D, D)
        self.resp_ = None                # responsibilities (N, K)
        self.n_iter_ = 0
        self.lower_bound_ = -np.inf


    def fit_predict(self, X, y=None):
        #this function returns a cluster number to each element of X
        raise NotImplementedError()

```

Questions 3

3.1 Use a subset of MNIST consisting of 2000 randomly sampled instances of "3"s and "7"s from the previous task. Apply PCA to reduce dimensionality. Implement GMM model from scratch (do not use *sklearn*) with random initialization of the parameters.

3.2 Perform the algorithm with K = 2, 3, ... , 10 and plot the corresponding objection function $\mathbb{L}$. Explain what you observe.

3.3 Build confusion matrices using *sklearn* for k-means and GMM for K=2 (both with PCA). Describe the results.

3.4 Describe the differences between k-means and GMM (deterministic/probabilistic, parameters, functions to optimize, updates).

3.5 Describe how k-means can be seen from the perspective of the EM-algorithm (*hint:* distance instead of the probability function).

Please provide your `Prénom_Nom.ipynb` file (replace `Prénom` and `Nom` with your real first and last name) including all **cell outputs**, and upload it on https://moodle.unige.ch before **Sunday, November 2, 2025, 21:59**.

Make sure that your notebook runs correctly from start to finish (use "Runtime/Restart session and run all"). If it fails to execute properly, it will have a **substantial negative impact on your grade**.

The assessment will mainly be based on your report, which must include your answers to all questions and the corresponding experimental results. *Particular importance is given to the mathematical explanations of your work, and your code should be clearly commented.*